ImageViewer コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

ImageViewerは、画像ファイル（JPG、PNG、GIF、BMP、WebP）を表示し、拡大・縮小・リセット機能を提供するビューアコンポーネントです。署名付きURLを使用してS3から安全に画像を取得・表示します。

1.2 使用場面

- プロジェクト管理画面で画像ファイルを選択した際の表示
- 実験結果として保存された画像ファイルの閲覧
- フローティングダイアログ内での画像表示

1.3 アクセス権限

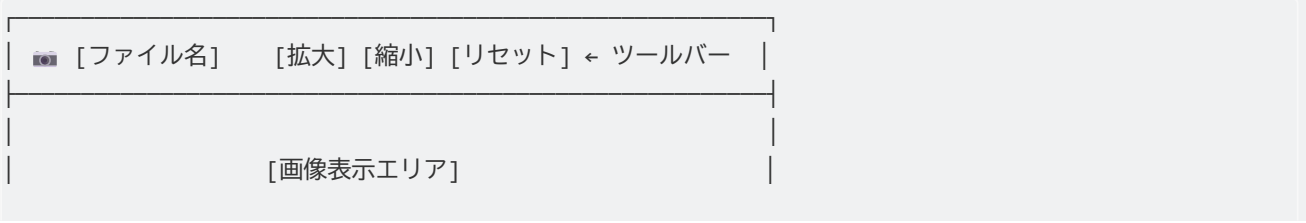
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時



```
│                  transform: scale()                    │
│                                                        │
│  ┌──────────────────────────────────────────────┐      │
│  │ サイズ: 1920 x 1080 px                       │      │
│  │ ズーム倍率: 1.20x                            │      │
│  │ ファイルキー: ...                            │      │
│  └──────────────────────────────────────────────┘      │
└────────────────────────────────────────────────────────┘
```

2.1.2 ロード中

```
┌────────────────────────────────────────────────────────┐
│ 📷 [ファイル名]    [拡大] [縮小] [リセット] (無効化)    │
├────────────────────────────────────────────────────────┤
│                                                        │
│                    🔄 Spinner                          │
│                  画像を読み込み中...                   │
│                                                        │
└────────────────────────────────────────────────────────┘
```

2.1.3 エラー時

```
┌────────────────────────────────────────────────────────┐
│ 📷 [ファイル名]    [拡大] [縮小] [リセット] [再読み込み] │
├────────────────────────────────────────────────────────┤
│                                                        │
│  ⚠️ エラー                                             │
│  画像の読み込みに失敗しました。                        │
│  サポートされていない形式かもしれません。              │
│  [再試行] ボタン                                       │
│                                                        │
└────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ツールバー

- **左側**: ファイルアイコン（FileImageOutlined、赤色 #CF0027）+ ファイル名（Title level 5）
- **右側**: 操作ボタン群
  - **拡大ボタン**: ZoomInOutlined アイコン

- **縮小ボタン**: ZoomOutOutlined アイコン
- **リセットボタン**: ReloadOutlined アイコン
- **再読み込みボタン**: Primary、エラー時のみ表示

2.2.2 画像表示エリア

- **背景色**: #f5f5f5（グレー）
- **配置**: 中央揃え（flex: center）
- **スクロール**: overflow: auto
- **画像**: `<AntImage>` コンポーネント、transform: scale()でズーム
- **プレビュー機能**: Ant Design Imageのプレビューモーダル

2.2.3 画像情報カード

- **サイズ**: 横幅 x 縦幅 px
- **ズーム倍率**: scale値（小数点2桁）
- **ファイルキー**: S3ファイルパス

---

3. 機能仕様

3.1 画像の読み込み

3.1.1 処理フロー

```
useEffect(() => {
  if (!fileKey) return;

  let isMounted = true;

  const load = async () => {
    setLoading(true);
    setError(null);
    setImageUrl(null);
```

```
    try {
      console.log('🔍 [ImageViewer] 署名付きURLを取得中:', fileKey);
      const url = await getFileViewUrl(fileKey);
      if (!isMounted) return;

      // 画像読み込み確認
      const img = new Image();
      img.onload = () => {
        if (!isMounted) return;
        console.log('✅ [ImageViewer] 画像読み込み成功:', img.width, 'x', img.height);
        setImageUrl(url);
        setImageDimensions({ width: img.width, height: img.height });
        setLoading(false);
      };
      img.onerror = (e) => {
        if (!isMounted) return;
        console.error('❌ [ImageViewer] 画像読み込みエラー:', e);
        setError('画像の読み込みに失敗しました。サポートされていない形式かもしれません。');
        setLoading(false);
      };
      img.src = url;
    } catch (err) {
      if (!isMounted) return;
      console.error('❌ [ImageViewer] URL取得エラー:', err);
      setError(`画像URLの取得に失敗しました: ${err instanceof Error ? err.message : '不明なエラー'}`);
      setLoading(false);
    }
  };

  load();

  return () => {
    isMounted = false;
  };
}, [fileKey, reloadTrigger]);
```

3.1.2 UI動作

- **開始時**: loadingフラグをtrueに設定、Spinner表示
- **URL取得**: `getFileViewUrl(fileKey)` で署名付きURL取得
- **画像検証**: `new Image()` で実際に読み込み、サイズを取得
- **成功時**: imageUrlとimageDimensionsを設定、画像表示
- **エラー時**: errorメッセージを設定、Alertとエラーボタン表示

3.1.3 再読み込み機能

- `reloadTrigger` Stateをインクリメントすることで再読み込み
- useEffectの依存配列に含まれるため、値が変わると再実行

3.2 ズーム操作

3.2.1 拡大（Zoom In）

```
const handleZoomIn = useCallback((): void => {
  setScale(prevScale => Math.min(prevScale * 1.2, 5));
}, []);
```

- **倍率**: 現在の1.2倍
- **最大**: 5倍まで

3.2.2 縮小（Zoom Out）

```
const handleZoomOut = useCallback((): void => {
  setScale(prevScale => Math.max(prevScale / 1.2, 0.1));
}, []);
```

- **倍率**: 現在の1/1.2倍
- **最小**: 0.1倍まで

3.2.3 リセット

```
const handleReset = useCallback((): void => {
  setScale(1);
}, []);
```

- **倍率**: 1倍（元のサイズ）に戻す

3.2.4 画像への適用

```
<AntImage
  src={imageUrl}
  style={{
    transform: `scale(${scale})`,
    transformOrigin: 'center center',
    maxWidth: '100%',
    maxHeight: '100%',
    objectFit: 'contain'
  }}
/>
```

- CSS `transform: scale()` で拡大・縮小
- `transformOrigin: 'center center'` で中心を基準に拡大

3.3 プレビューモーダル

```
<AntImage
  preview={{
    mask: 'プレビュー'
  }}
/>
```

- Ant Design Imageのプレビュー機能を有効化
- 画像クリックでフルスクリーンモーダル表示
- モーダル内で独自のズーム・回転機能が利用可能

---

4. データ仕様

4.1 Props定義

```
interface ImageViewerProps {
  fileKey: string;
  fileName: string;
}
```

Props説明:

Prop名	型	必須	説明
fileKey	string	✅	S3ファイルキー（署名付きURL取得に使用）
fileName	string	✅	表示するファイル名

4.2 State定義

```
const [imageUrl, setImageUrl] = useState<string | null>(null);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);
const [scale, setScale] = useState<number>(1);
const [reloadTrigger, setReloadTrigger] = useState<number>(0);
const [imageDimensions, setImageDimensions] = useState<{ width: number; height: number }>({
  width: 0,
  height: 0
});
```

State説明:

State名	型	初期値	説明
imageUrl	string | null	null	署名付きURL
loading	boolean	true	ロード中フラグ
error	string | null	null	エラーメッセージ
scale	number	1	ズーム倍率（1 = 100%）
reloadTrigger	number	0	再読み込みトリガー（インクリメントで再読み込み）
imageDimensions	{ width, height }	{ width: 0, height: 0 }	画像の元サイズ（px）

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
1	getFileViewUrl	-	-	✅ 実装済み	S3署名付きURL取得（fileAccessService）

5.2 API呼び出しコード

```
const url = await getFileViewUrl(fileKey);
```

備考:

- `getFileViewUrl` は `fileAccessService` で定義されたサービス関数
- バックエンドAPIを呼び出してS3署名付きURLを取得
- 有効期限付きURL（通常1時間）

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
署名付きURL取得失敗	errorメッセージ設定、Alert表示、再読み込みボタン表示
画像読み込み失敗（img.onerror）	エラーメッセージ設定、Alert表示、再試行ボタン表示
サポートされていない画像形式	「サポートされていない形式かもしれません」メッセージ表示

6.2 エラー処理コード

6.2.1 URL取得エラー

```
try {
  const url = await getFileViewUrl(fileKey);
  // ...
} catch (err) {
  if (!isMounted) return;
  console.error('❌ [ImageViewer] URL取得エラー:', err);
  setError(`画像URLの取得に失敗しました: ${err instanceof Error ? err.message : '不明なエラー'}`);
  setLoading(false);
}
```

6.2.2 画像読み込みエラー

```
img.onerror = (e) => {
  if (!isMounted) return;
  console.error('❌ [ImageViewer] 画像読み込みエラー:', e);
  setError('画像の読み込みに失敗しました。サポートされていない形式かもしれません。');
  setLoading(false);
};
```

エラーメッセージ:

- "画像URLの取得に失敗しました: [エラーメッセージ]"
- "画像の読み込みに失敗しました。サポートされていない形式かもしれません。"

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/ImageViewer.tsx`

7.2 関連ファイル

- **サービス**: `frontend/src/services/fileAccessService.ts` (getFileViewUrl)

7.3 依存コンポーネント

- `<AntImage />` - Ant Design Imageコンポーネント（プレビューモーダル付き）

7.4 依存ライブラリ

- `antd` : Card, Button, Space, Typography, Alert, Spin, Flex, Image
- `@ant-design/icons` : ZoomInOutlined, ZoomOutOutlined, ReloadOutlined, FileImageOutlined, LoadingOutlined

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline